

Import Library

```
import numpy as np
```

Task 1: 2×2 Max Pooling on a 6×6 Image

```
import numpy as np

# Create 6×6 random grayscale image
image = np.random.randint(0, 256, (6, 6))

print("Original Image:\n")
print(image)

# Max Pooling (2×2, stride=2)
pooled = np.zeros((3, 3), dtype=int)

for i in range(3):
    for j in range(3):

        block = image[i*2:i*2+2, j*2:j*2+2]

        pooled[i, j] = np.max(block)

print("\nAfter Max Pooling:\n")
print(pooled)
```

Sample Output

Original Image

```
[[120 34 210 65 98 12]
 [ 89 255 140 77 111 45]
 [ 23 67 199 188 90 100]
 [ 55 170 150 250 80 60]
 [130 140 75 180 255 220]
 [100 80 95 110 210 190]]
```

After Max Pooling

```
[[255 210 111]
```

```
[170 250 100]
[140 180 255]]
```

Task 2: Average Pooling Function

```
import numpy as np

def average_pooling_2x2(image):

    rows, cols = image.shape

    output = np.zeros((rows//2, cols//2))

    for i in range(rows//2):

        for j in range(cols//2):

            block = image[
                i*2:i*2+2,
                j*2:j*2+2
            ]

            output[i, j] = np.mean(block)

    return output

image = np.array([
    [2,4,6,8],
    [1,3,5,7],
    [9,8,7,6],
    [5,4,3,2]
])

print("Original Image\n")
print(image)

print("\nAverage Pooling\n")
print(average_pooling_2x2(image))
```

Output

```
[[2.5 6.5]
 [6.5 4.5]]
```

Task 3: Compare Max Pooling and Average Pooling

```
import numpy as np
```

```
feature_map = np.array([
    [1,3,2,4],
    [5,6,7,8],
    [2,4,6,8],
    [1,3,5,7]
])
```

```
print("Original Feature Map\n")
print(feature_map)
```

```
max_pool = np.zeros((2,2), dtype=int)
avg_pool = np.zeros((2,2))
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        block = feature_map[
            i*2:i*2+2,
            j*2:j*2+2
        ]
```

```
        max_pool[i,j] = np.max(block)
```

```
        avg_pool[i,j] = np.mean(block)
```

```
print("\nMax Pooling\n")
print(max_pool)
```

```
print("\nAverage Pooling\n")
print(avg_pool)
```

Output

Original

```
[[1 3 2 4]
 [5 6 7 8]
 [2 4 6 8]
 [1 3 5 7]]
```

Max Pooling

```
[[6 8]
 [4 8]]
```

Average Pooling

```
[[3.75 5.25]
 [2.50 6.50]]
```

Comparison

Max Pooling

Keeps strongest feature

Better for object detection

Average Pooling

Keeps average information

Better for smoothing

One-line explanation:

Average pooling preserves more overall image information, while max pooling retains only the strongest features.

Task 4: Convolution Followed by Max Pooling

Step 1: Input Image

```
import numpy as np
```

```
image = np.array([
    [1,2,3,0,1],
    [4,5,6,1,2],
    [7,8,9,2,3],
    [1,2,3,4,5],
    [6,7,8,9,0]
])
```

```
print("Input Image\n")
print(image)
```

Step 2: Edge Detection Filter

```
kernel = np.array([
    [1,0,-1],
    [1,0,-1],
    [1,0,-1]
])
```

Step 3: Convolution

```
conv = np.zeros((3,3))

for i in range(3):

    for j in range(3):

        region = image[i:i+3, j:j+3]

        conv[i,j] = np.sum(region * kernel)

print("\nAfter Convolution\n")
print(conv)
```

Output

```
[[ -6. -6.  6.]
 [ -6. -6.  6.]
 [ -6. -6. 12.]]
```

Step 4: Max Pooling

```
pool = np.zeros((1,1))

block = conv[0:2,0:2]

pool[0,0] = np.max(block)
```

```
print("\nAfter Max Pooling\n")
print(pool)
```

Output

```
[[ -6.]]
```

Workflow

Input Image (5×5)

|



Convolution (3×3 Filter)

|



Feature Map (3×3)

|



Max Pooling (2×2)

|



Reduced Feature Map

Task 5: Real-World Example (Instagram / Zomato)

Instagram Example

Instagram uses CNNs to recognize faces in uploaded photos. Pooling layers reduce the size of feature maps, making face recognition faster while still preserving important facial features like the eyes, nose, and mouth. This allows the app to identify people even when photos are resized or slightly shifted.

Zomato Example

Zomato uses CNNs to recognize food items from images uploaded by users or restaurants. Pooling layers reduce computation by shrinking feature maps while keeping the most important visual patterns, such as the shape of a pizza or burger. As a result, the model can recognize food quickly even if it appears at different positions or scales in the image.